You should also be aware that assignments made within the `PageModel` will be overridden if it is also set in the Razor page itself.

Another way to access `ViewData` values is via a property called `ViewBag`. This property is a wrapper around `ViewData` that enables you to treat it as a dynamic object. Consequently, you access items through property names that match their key. Because the values are dynamic, there is no need to explicitly cast when working with non-string types:

```
@ViewBag.SaleEnds.ToString("h tt, MMMM dd")
```

`ViewBag` is only available in Razor files. It is not available in the `PageModel` class, although it is available in MVC controllers, where it was carried over from an old framework called ASP.NET Web Pages. The ASP.NET Core team decided not to implement `ViewBag` in Razor Pages `PageModel` classes because, being dynamic, it produced a measurable performance impact on views and pages that used it. For that reason, use of `ViewBag` is not recommended at all.

`ViewData` is weakly typed, so you get no compile-time checking or IntelliSense support. It relies on referencing items using a string-based key—an approach that is sometimes referred to as *working with magic strings*. Consequently, working with `ViewData` can cause errors because it is so easy to mistype a string or rename it in one place and forget to update other references. If you try to render an item that doesn't exist, nothing is rendered. If you attempt to call a method on a `ViewData` entry that doesn't exist, a `NullReferenceException` will be raised. If you attempt to cast a `ViewData` entry to the wrong type, you will generate an `InvalidCastException`. These exceptions will occur at run time.

`ViewData` itself should only be used sparingly. It is useful for passing small pieces of simple data to layout pages, such as the page's title. For all other scenarios in which you need to pass data from a `PageModel` class to a Razor page, you should use public `PageModel` properties, which we will look at next.